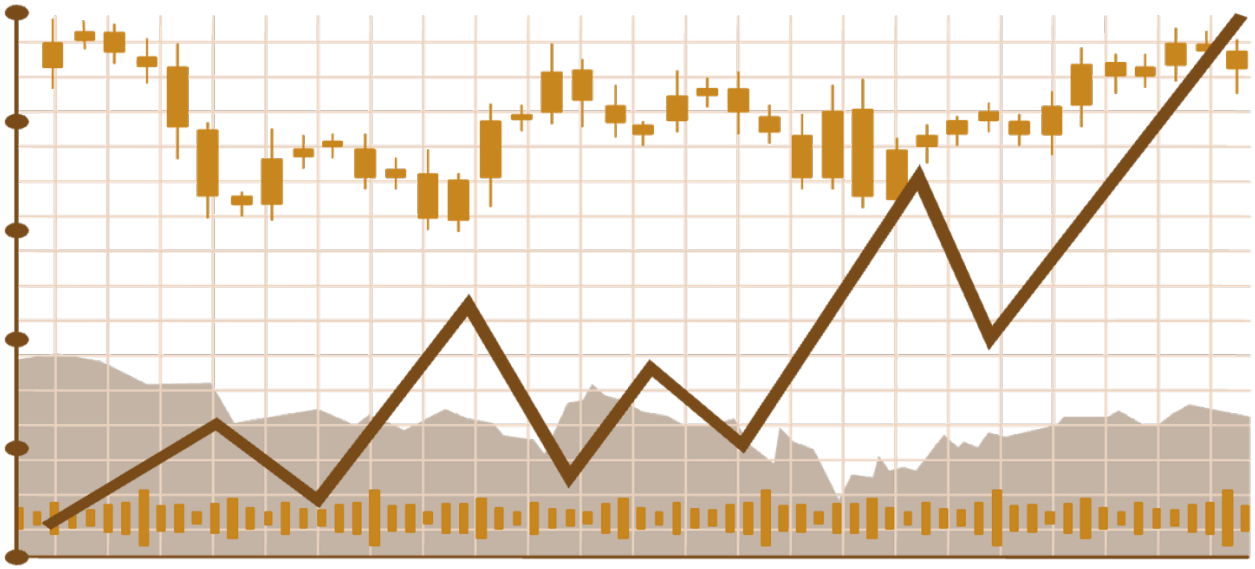
TSM-MachLeData
Beat Wolf
Bertil Chapuis
S1 - 2025-2026

Hes·so



Stock Price Prediction using Machine Learning

Advanced MLOps Topic: Data Drift

Justin Ferrara **Luca Da Silva** **Rafael Cardoso** **Thomas Maksimous**

09.12.2025

Table of contents

1 - Introduction 3
2 - Advanced Topic - Data Drift 4
3 - Project Scoping 4
    3.1 - Objectives 4
    3.2 - Requirements 5
    3.3 - Project Constraints and Associated Risks 5
4 - Fisrt Version 5
    4.1 - Data Engineering 5
    4.2 - ML Model Development 6
    4.3 - Deployment 7
5 - Performance Improvements 8
    5.1 - Feature Engineering Enhancements 8
    5.2 - Model Architecture Optimization 8
    5.3 - Results 8
6 - Data Drift Analysis and Integration 9
    6.1 - Prepared Step Improvements 9
    6.2 - Pipeline Enhancements 9
    6.3 - Data Drift Detection Strategy 10
    6.4 - Monitoring Approach 10
    6.5 - Results and Findings 10
7 - Conclusion 11
Bibliography 12

1 - Introduction

The objective is to create a simple predictive market model. The model would take as input the stock closing price for each month, with the goal of predicting the closing prices for the next 5 days. The dataset would consist of the past 5 year's data for the most popular technology stocks in the NASDAQ index.

As an advanced MLOps problem, we will study the data drift in our results. The goal is to fine-tune our model every week using the new data from the previous week. We will then compare the performance of our models over time. The objective here is to observe whether the performance of older models degrades over time compared to their updated fine-tuned versions. (Model drift)

TODO more references and go deeper about the results obtained

2 - Advanced Topic - Data Drift

In machine learning systems, **model drift** (also known as concept drift) refers to the degradation of a model's predictive performance over time due to changes in the underlying data distribution or relationships between input and output variables. This phenomenon occurs because the model was trained on past data that may no longer represent current real-world conditions. In financial forecasting, for instance, market trends, investor behavior, or economic indicators may evolve, causing previously learned patterns to become outdated. Detecting and managing model drift is therefore critical to maintaining accuracy and reliability in predictive systems.

One key source of model drift is **data drift**, which occurs when the statistical properties of input features (the X variables) change over time. For example, if stock trading volumes or volatility levels shift due to external economic events, the distribution of input data no longer aligns with what the model originally learned. Data drift can be measured through statistical tests such as the Kolmogorov-Smirnov test or by comparing feature distributions between training and production data. Regular monitoring allows to identify when the input data environment has shifted significantly, prompting the need for model retraining or adjustment.

A related but distinct phenomenon is **feature drift**, which occurs when the relevance or predictive power of certain input features changes relative to others. For example, a feature like "daily trading volume" might become less predictive of price movement if market liquidity conditions change. Feature drift affects model interpretability and can mislead the model's weighting of inputs, even if overall data distributions appear stable. Detecting feature drift often involves monitoring metrics like feature importance or mutual information across time.

Finally, **label drift** (or target drift) refers to a change in the distribution of the target variable (Y). In stock prediction, this could occur if the overall market sentiment changes and the expected returns follow a different trend than before. Label drift can appear independently of input changes. For instance, when a company changes its dividend policy or experiences sudden external shocks. Continuous evaluation on recent labeled data helps detect this drift and guides the decision to retrain or recalibrate the model.

Together, these forms of drift—model, data, feature, and label, highlight the importance of dynamic monitoring and adaptation in MLOps. Properly managing them ensures that machine learning models remain accurate, fair, and robust in evolving real-world environments.

TODO more references and go deeper

[1], [2], [3], [4], [5]

3 - Project Scoping

The project aims to develop a machine learning model to predict stock opening prices, while implementing a comprehensive MLOps lifecycle to ensure the model's robustness and adaptability over time.

3.1 - Objectives

The objective of this project is to predict the opening stock value of different companies using machine learning techniques. The purpose is to provide useful insights to investors, helping them make more informed and data-driven investment decisions. The key problem to solve is the accurate prediction of stock prices, which varies due to market volatility and numerous influencing factors.

The advanced MLOps topic covered in this project will be Model Drift detection and mitigation, more specifically the Data Drift and Feature Drift aspects. In this context, we will monitor how changes in the input data distribution over time affect the model's performance. The main goal is to maintain the model's reliability and adaptability, ensuring it remains at its best performance level even as market conditions evolve.

3.2 - Requirements

This project will require the following resources:

- Financial datasets containing daily stock data (open, close, high, low, volume, etc.) with sufficient historical depth.
- Development environment for data science and MLOps:
  - Python (pandas, scikit-learn, TensorFlow/PyTorch)
  - DVC or MLflow for experiment tracking and versioning
  - Docker and GitHub Actions for CI/CD
  - Cloud or local deployment environment (e.g., Google Cloud, Azure, or Kubernetes cluster)

Monitoring tools to track model drift (e.g., Evidently AI, Prometheus + Grafana, or custom scripts).

3.3 - Project Constraints and Associated Risks

The project operates under multiple constraints and risks that could influence its success. Time limitations may restrict the extent of model experimentation and deployment testing, while limited financial domain expertise could block accurate interpretation of market-related outcomes. Additionally, access to reliable financial data may be limited or require paid sources, and the technical complexity of implementing MLOps processes such as continuous integration, monitoring, and retraining adds further challenges.

These constraints comes with risks including potential costs associated with cloud technologies and APIs, the inherent instability of models due to unpredictable market dynamics, and the possibility of overfitting, which could reduce the model's ability to generalize effectively to unseen market conditions.

4 - Fisrt Version

TODO cite [6]

4.1 - Data Engineering

The data engineering phase is crucial for the success of the stock price prediction project. It involves several key steps to ensure that the data is properly collected, processed, and stored for effective use in machine learning models.

4.1.1 - Data Acquisition

To acquire the necessary data for stock price prediction, we used the https://www.nasdaq.com website to download historical stock data for 20 stocks in the NASDAQ index. The data collected is in CSV format, and contains columns such as Date, Close, Open, High, Low, and Volume for the past 5 years. The following companies were selected for this project:

- Apple Inc. (AAPL)
- AMD Inc. (AMD)
- Amazon.com, Inc. (AMZN)
- ASML Holding N.V. (ASML)
- Broadcom Inc. (AVGO)
- Cisco Systems, Inc. (CSCO)
- Alphabet Inc. (GOOGL)
- International Business Machines Corporation (IBM)
- Facebook, Inc. (META)
- Microsoft Corporation (MSFT)
- Micron Technology, Inc. (MU)
- Netflix, Inc. (NFLX)
- NVIDIA Corporation (NVDA)
- Oracle Corporation (ORCL)

- Palantir Technologies Inc. (PLTR)
- Qualcomm Incorporated (QCOM)
- Starbucks Corporation (SBUX)
- Tencent Holdings Ltd. (TCEHY)
- Tesla, Inc. (TSLA)
- Taiwan Semiconductor Manufacturing Company Ltd. (TSM)

The data spans from 11/10/2020 to 11/07/2025 for each stocks, providing a comprehensive dataset for training and evaluating the machine learning models.

4.1.2 - Data Preprocessing

The preprocessing steps involved cleaning the data and transforming the data into a suitable format for modeling. We performed the following preprocessing tasks:
- Checking for missing values and handling them appropriately (e.g., imputation or removal).
- Converting the closing prices to float data variables, representing the variation in percent from the previous closing price.
- Normalizing the data to ensure that all features are on a similar scale, which is important for machine learning algorithms.
- Splitting the data into training, validation, and test sets to evaluate the model's performance effectively.

4.1.3 - Data Storage

The raw and preprocessed data are stored in a structured format using CSV files and DVC for version control. This allows us to track changes in the data over time and ensures reproducibility in our experiments. The data is stored in a dedicated S3 bucket for easy access and scalability (as shown in the tutorial https://mlops.swiss-ai-center.ch/).

4.2 - ML Model Development

To develop the machine learning model for stock price prediction, we followed the https://mlops.swiss-ai-center.ch/ guide to set up our MLOps pipeline. This involved several key steps to ensure that the model is trained, validated, and deployed effectively.

4.2.1 - Feature Engineering

TODO

4.2.2 - Model Training and Validation

The model is trained using the preprocessed data, which includes the stock closing prices and other relevant features. We used LSTM (Long Short-Term Memory) neural networks, which are well-suited for time series data like stock prices. The training process involves optimizing the model's parameters to minimize the prediction error on the training set.

The validation set is used to tune hyperparameters and prevent overfitting.

TODO more details

4.2.3 - Model Evaluation

The model's performance is evaluated using metrics such as Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R-squared (R^2). These metrics provide insights into how well the model predicts stock prices compared to actual values. We also perform backtesting to assess the model's performance over historical data.

Here is a brief explanation of the evaluation metrics used:

- The Mean Absolute Error (MAE) measures the average absolute difference between predicted and actual values, showing how far predictions are from reality on average.
- The Root Mean Squared Error (RMSE) calculates the square root of the average squared errors, giving more weight to larger mistakes and highlighting significant prediction deviations.
- The R-squared (R^2) indicates how well the model explains the variance in the actual data, with values closer to 1 representing better predictive accuracy.

The best and worst predictions are visualized using plots to illustrate the model's performance and to better understand its strengths and weaknesses.

4.3 - Deployment

To deploy the model, we use the https://mlops.swiss-ai-center.ch/ guidelines to set up a base environment for deployment.

The model is trained locally. On every pull request, the model is retrained in a Kubernetes pod using GitHub Actions. The trained model is then containerized using Docker and deployed to a cloud environment (e.g., AWS, GCP, or Azure) for scalability and accessibility. This setup correspond to the Chapter 3.7 of the MLOps guide.

TODO

4.3.1 - CI/CD Pipeline

The CI/CD pipeline, defined in the MLOps workflow, automates the entire lifecycle of model development, testing, and deployment using GitHub Actions, DVC, CML, and BentoML. It runs on a self-hosted runner for training tasks and on GitHub-hosted runners for deployment.

The pipeline is triggered automatically when:
- Code is pushed to the main branch,
- A pull request is opened, or
- It is manually triggered via the GitHub Actions interface.

It consists of two main jobs:
1. Train and Report (train-and-report)

   Triggered on pull requests, this job automates model training and performance reporting:
   - Checks out the repository and sets up `Python 3.12`.
   - Installs dependencies and authenticates with Google Cloud.
   - Reproduces the model training pipeline using DVC (`dvc repro --pull`).
   - Pushes updated model artifacts and metrics to the DVC remote storage.
   - Automatically commits changes to `dvc.lock`.
   - Generates a CML report comparing the new experiment results to the main branch, including:
     - Parameter and metric differences (dvc params diff, dvc metrics diff),
     - Visual comparisons of plots (training history, prediction previews, and confusion matrices).
   - Publishes the report as a comment in the pull request for review.
2. Publish and Deploy (publish-and-deploy)

   Runs when changes are merged into the main branch. It performs the following steps:
   - Checks out the repository and installs dependencies.
   - Authenticates with Google Cloud and reproduces the final model pipeline using DVC.
   - Imports and builds the BentoML model artifact.
   - Containerizes the Bento model and pushes Docker images to the Google Container Registry.
   - Retrieves credentials for the Google Kubernetes Engine (GKE) cluster.
   - Updates the Kubernetes deployment manifest to reference the new image version (based on the Git SHA).

- Applies the updated deployment and service configuration to deploy the new model version to Kubernetes.

4.3.2 - Infrastructure

The infrastructure for deploying the machine learning model is set up using Google Cloud Platform (GCP). We utilize Google Kubernetes Engine (GKE) to manage the containerized application, ensuring scalability and reliability. The model is containerized using Docker, allowing for consistent deployment across different environments.

4.3.3 - Scalability

As the model is deployed on GKE, it can easily scale based on demand. Kubernetes allows us to define resource limits and auto-scaling policies to handle varying workloads. This ensures that the application remains responsive even during peak usage times.

5 - Performance Improvements

As seen in the previous section, the initial model[6] was not performing well, with high error rates and low predictive accuracy. To address these issues, we implemented several performance improvements to enhance the model's capabilities, described in detail in PR [7].

5.1 - Feature Engineering Enhancements

To improve the model's performance, we extended the input data to include the past 60 days of stock prices instead of just 30 days. This provided the model with more historical context, allowing it to capture longer-term trends and patterns in stock price movements.

We also added new features derived from the existing data, to provide additional insights for the model. These features are described in the following table:

- **Short-term trend indicators:** Computed 5-day and 10-day simple moving averages (SMA_5, SMA_10) to smooth short-horizon fluctuations and capture underlying price trends.
- **Recent-information-weighted trend measure:** Incorporated a 5-day exponential moving average (EMA_5) to assign greater weight to recent observations and enhance sensitivity to short-term market movements.
- **Volatility approximation:** Derived a High-Low price spread feature as a proxy for intraday volatility.
- **Directional price-movement ratio:** Constructed an Open-to-Close price ratio to quantify daily directional bias.
- **Volume-based activity metric:** Calculated the percentage change in trading volume to characterize shifts in market activity.

5.2 - Model Architecture Optimization

To avoid overfitting and improve generalization, we increased the dropout rate in the LSTM layers from 0.1 to 0.2. This adjustment helped the model to better handle unseen data by preventing it from relying too heavily on specific neurons during training.

5.3 - Results

The performance improvements led to significant enhancements in the model's predictive accuracy and reliability. The key results are summarized in the pull request [7], which includes:

- A reduction in Mean Absolute Error (MAE) from 1.87 to 0.66.
- An decrease in R2 Score from 0.00 to −1.19 (which is not an improvement, but expected due to the nature of stock price prediction).

- Visualizations comparing the model's predictions before and after the improvements, demonstrating better alignment with actual stock prices.

6 - Data Drift Analysis and Integration

To strengthen the robustness of the stock-prediction workflow, data drift detection was integrated directly into the MLOps pipeline with pull request [8]. The `evidently` package was added as the primary toolkit for measuring drift on the prepared datasets [9]. The package `evidently` was chosen due to its comprehensive suite of drift detection methods, base on multiple algorithms depending on the column type [2].

6.1 - Prepared Step Improvements

The data preparation stage was expanded to output two parallel datasets:

- Candidate data: the newly prepared dataset generated from the latest weekly ingestion (from folder `data/raw_candidate/`).
- Current data: the previously accepted dataset used for training the active model (from folder `data/raw/`).

These datasets are stored in `data/prepared/` and `data/prepared_candidate/` respectively, allowing for direct comparison of feature distributions over time.

6.2 - Pipeline Enhancements

A new data drift detection step was added to the workflow. This component:

- Loads the current and candidate prepared datasets.
- Uses evidently to compute drift metrics across all features.
- Generates:
  - An HTML report for human-readable inspection.
  - A JSON report containing structured drift statistics suitable for automated analysis.
- Stores both reports as pipeline artifacts.
- Inserts a summary of the JSON report into pipeline comments for quick visibility during CI/CD reviews.

This approach ensures that every potential model update is automatically preceded by a drift evaluation that highlights feature-level changes.

The pipeline comment summarize the key findings from the JSON report, including:
- If a significant drift was detected for any features.
- The number of drifted columns (features).
- The number of columns analyzed (features).

An example of the pipeline comment is shown in pull request [8]. The Figure 1 shows the generated report from the `Evidently` package on the original data without drift.

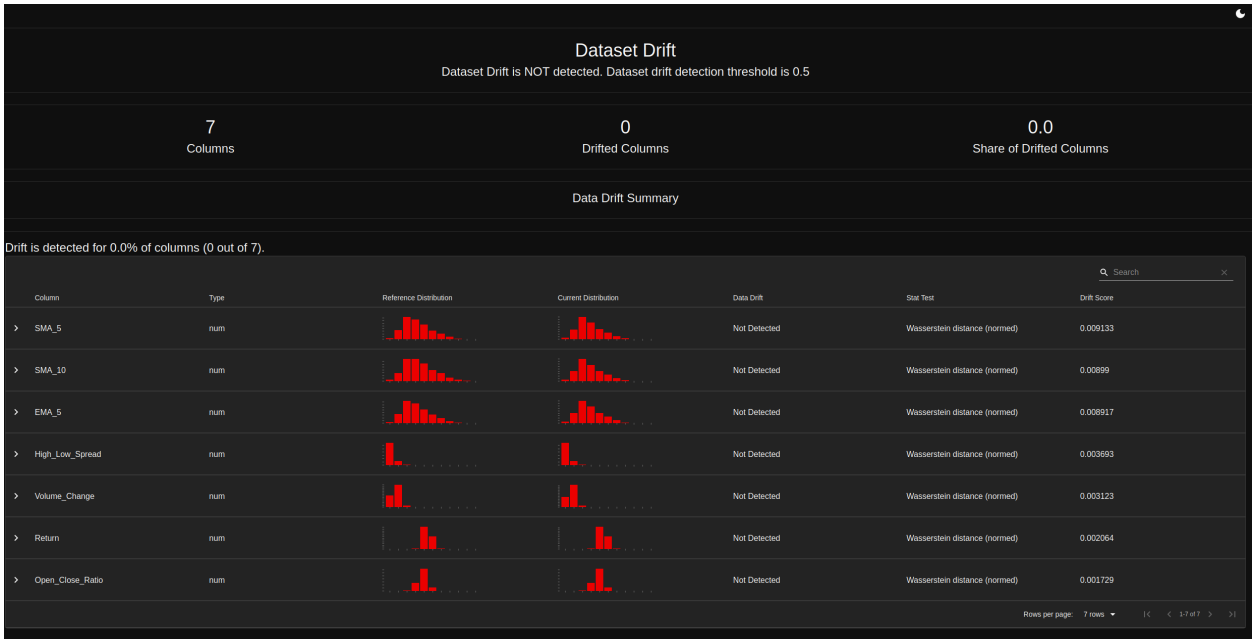


Figure 1 - Data Drift Report Example from Evidently

6.3 - Data Drift Detection Strategy

The first step in our data drift detection strategy is to compare the statistical distributions of features between the current and candidate datasets. Using evidently's built-in statistical tests (e.g., Kolmogorov-Smirnov test for continuous features, Chi-squared test for categorical features), we assess whether significant changes have occurred in the input data over time.

The second step to detect data drift involves a dual-model strategy within the pipeline. The idea is to train and evaluate two models in parallel: a base model that remains fixed over time (based on initial data), and a new model, trained with the latest data (`candidate` model) on a weekly basis.

By comparing the performance difference between these two models over time, we can infer whether drift is present. When the fine-tuned model consistently outperforms the fixed base model on new data, it signals that the distribution has shifted and that continuous retraining is necessary for optimal predictive accuracy.

6.4 - Monitoring Approach

Monitoring is performed exclusively through the pipeline rather than a dedicated external monitoring service. This setup enables:

- Regular evaluation of drift through evidently
- Tracking performance of both base and fine-tuned models
- Automated generation of drift reports as part of routine CI/CD operations

Although this approach ensures consistent oversight during the weekly data refresh cycle, it does not offer real-time monitoring or the granular diagnostics provided by specialized observability platforms. Nonetheless, it remains a practical approach within the project scope and supports effective detection of data drift and early signs of model degradation.

6.5 - Results and Findings

The integration of data drift detection into the MLOps pipeline provided valuable insights into the stability of the input data over time. However, the analysis revealed that no significant data drift was detected between the current and candidate datasets. Both models—the fixed base model and the fine-tuned candidate model—performed similarly over time, indicating that the underlying data distribution

remained relatively stable during the evaluation period. This might be attributed to the nature of stock market data, which can exhibit periods of stability interspersed with volatility. The pull request [8] contains detailed results and visualizations from the drift analysis.

To have a better comprehension of the data drift and to show a sample of the results obtained, a fake data drift has been implemented in branch [10]. This allows to see how the data drift detection works in practice and to validate the effectiveness of the implemented approach. The Figure 2 shows an example of data drift detected using the fake data drift implementation.

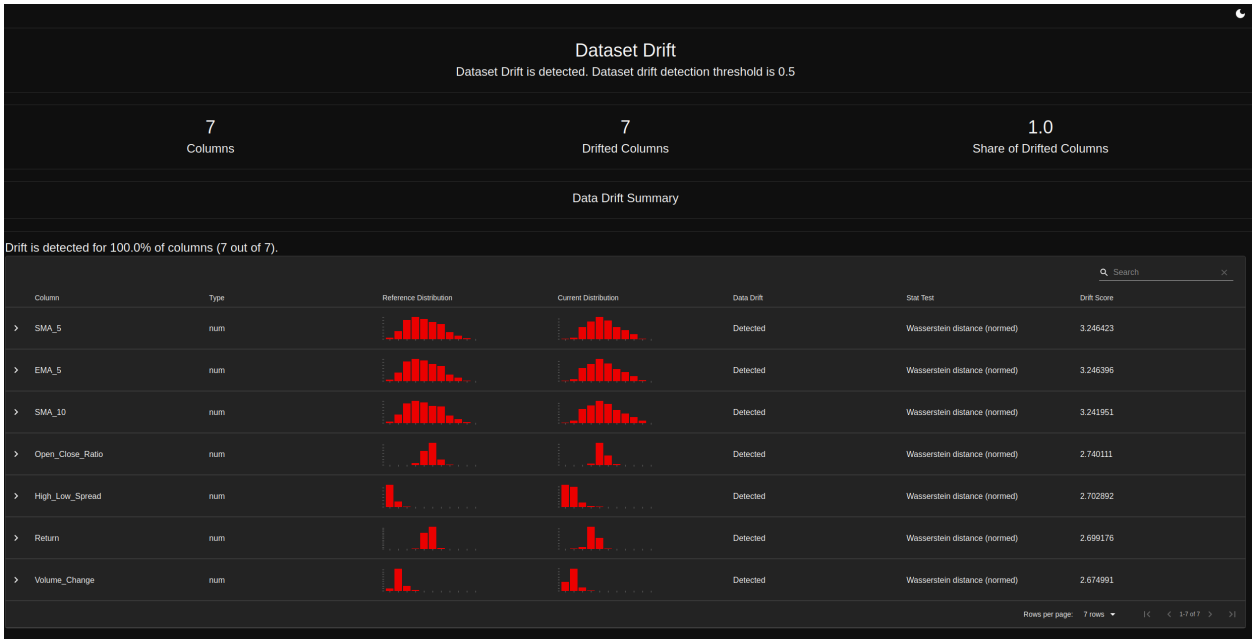


Figure 2 - Artificial Data Drift Report Example from Evidently

7 - Conclusion

Assessment ??

TODO

TODO speak about the limitations of the project and possible improvements (more data, more features, other models, etc.)

Bibliography

[1] Evidently AI, "What is data drift in ML, and how to detect and handle it." Accessed: Dec. 08, 2025. [Online]. Available: https://www.evidentlyai.com/ml-in-production/data-drift

[2] Evidently AI, "Data Drift - How data drift detection works." Accessed: Dec. 08, 2025. [Online]. Available: https://docs.evidentlyai.com/metrics/explainer_drift

[3] SymuFolk, "Detecting Data Drift - Tools, Techniques, and Best Practices for 2025." Accessed: Dec. 08, 2025. [Online]. Available: https://medium.com/@Symufolk/detecting-data-drift-tools-techniques-and-best-practices-for-2025-d0d34b863c3e

[4] Jayita Gulati, "Detecting & Handling Data Drift in Production." Accessed: Dec. 08, 2025. [Online]. Available: https://machinelearningmastery.com/detecting-handling-data-drift-in-production/

[5] Karyna Naminas, "Data Drift - Detection and Monitoring Techniques." Accessed: Dec. 08, 2025. [Online]. Available: https://labelyourdata.com/articles/machine-learning/data-drift

[6] Justin Ferrara, "Stock-ML Repository - Change to financial data PR 6." Accessed: Dec. 08, 2025. [Online]. Available: https://github.com/JustinFerrara14/Stock-ML/pull/6

[7] Justin Ferrara, "Stock-ML Repository - Performance Improvements PR 10." Accessed: Dec. 08, 2025. [Online]. Available: https://github.com/JustinFerrara14/Stock-ML/pull/10

[8] Justin Ferrara, "Stock-ML Repository - Data Drift Integration PR 12." Accessed: Dec. 08, 2025. [Online]. Available: https://github.com/JustinFerrara14/Stock-ML/pull/12

[9] Evidently AI, "Data Drift - Overview of the Data Drift Preset.." Accessed: Dec. 08, 2025. [Online]. Available: https://docs.evidentlyai.com/metrics/preset_data_drift

[10] Justin Ferrara, "Stock-ML Repository - Create Artificial Data Drift Branch." Accessed: Dec. 08, 2025. [Online]. Available: https://github.com/JustinFerrara14/Stock-ML/tree/10-create-artificial-datadrift

[11] Justin Ferrara, "Stock-ML Repository." Accessed: Dec. 08, 2025. [Online]. Available: https://github.com/JustinFerrara14/Stock-ML